WoSM Academy

CS102-F26 · DIGITAL DESIGN AND COMPUTER ARCHITECTURE I

Counters

Lecture 13: the synchronous up counter and the asynchronous ripple counter

Dr. Ali Khudiyev

Fall 2026

CC BY-SA 4.0

Today

01 Counting

02 Synchronous up counter

03 Asynchronous ripple counter

04 Summary

01

Counting

What a counter is

Definition (Counter)

An n-bit *counter* outputs an n-bit binary number and, at each clock edge, adds 1 to it (an *up counter*) or takes 1 away (a *down counter*).

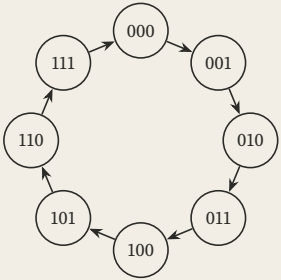
- 0… 1… 2… 3…: the state is the number shown so far.
- Between rising edges the output holds, as in every flip-flop.

clk	Q_t	Q^{up}_{t+1}	Q^{down}_{t+1}
↑	000	001	111
↑	001	010	000
↑	010	011	001
↑	011	100	010
↑	100	101	011
↑	101	110	100
↑	110	111	101
↑	111	000	110
0, 1, ↓	q	q	q

A 3-bit up and down counter (Table 5.15)

Wrapping around

- After 111 (7), the up counter goes back to 000. After 000, the down counter goes to 111.
- So a 3-bit up counter that starts at 0 shows k mod 8 after k edges; an n-bit one counts modulo 2^n.
- As a state machine `(Unit 10)`: 8 states in a ring, one step per clock edge, no other input. The output is the state.



Not a bug

The wrap-around is a feature. The DX8's stepper `(Unit 21)` is a 3-bit counter with a decoder: it goes through steps 1 to 8, then starts again at step 1.

02

Synchronous up counter

Which bits flip?

y_2	y_1	y_0	T_2	T_1	T_0
0	0	0	0	0	1
0	0	1	0	1	1
0	1	0	0	0	1
0	1	1	1	1	1
1	0	0	0	0	1
1	0	1	0	1	1
1	1	0	0	0	1
1	1	1	1	1	1

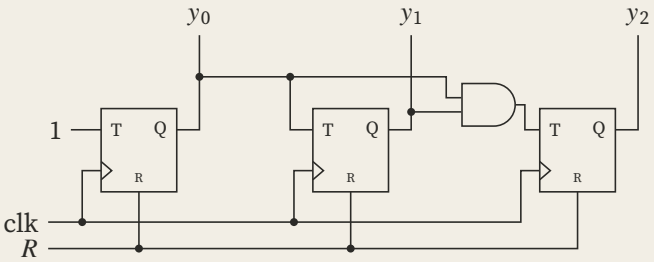
$T_i = 1$: bit i flips at the next edge

- Adding 1 in binary: bit i flips exactly when all the bits below it are 1 (the carry reaches it).
- A T flip-flop flips when $T = 1$ (Unit 12). So use one per bit, with

$$T_0 = 1, \qquad T_1 = y_0, \qquad T_2 = y_0\, y_1.$$

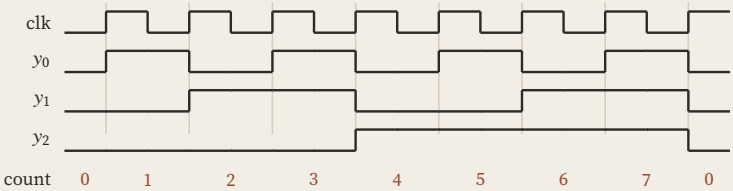
- In general, $T_i = y_0\, y_1 \cdots y_{i-1}$.

The circuit



- One clock for all three flip-flops: they all change at the same edges, in sync with the clock. Hence *synchronous* counter.
- R is the asynchronous reset of Lecture 12: $R = 1$ sets the count to 000.

In time



- All the bits that flip, flip together, right after a rising edge.
- y_0 has half the frequency of the clock, y_1 a quarter, y_2 an eighth: bit i divides the clock frequency by 2^{i+1}.

More bits

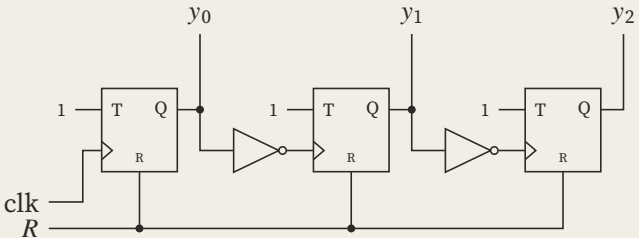
- A 4-bit counter adds $T_3 = y_0\, y_1\, y_2$; an 8-bit one goes on to $T_7 = y_0\, y_1 \cdots y_6$.
- Each T_i is the one before it, ANDed with one more bit: $T_i = T_{i-1}\, y_{i-1}$. A chain of AND gates.
- With D flip-flops instead, feed each one $D_i = y_i \oplus T_i$: that is $Q^+ = T \oplus Q$ again (Unit 12).

> **The longest path**
>
> After an edge, the new y_0 has to pass through the whole AND chain before T_{n-1} is ready for the next edge. Lecture 14 turns this into a limit on the clock period.

03

Asynchronous ripple counter

Clocked by the bit before



- Every T is 1: each flip-flop flips at each rising edge of *its own* clock. Only the first one gets the real clock; flip-flop i gets $\overline{y_{i-1}}$.
- $\overline{y_{i-1}}$ rises exactly when y_{i-1} falls from 1 to 0: when a carry moves up into bit i.

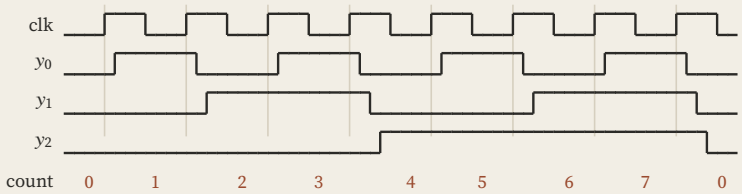
Step by step

	$t=0$	1	2	3	4	5	6	7	8
$\mathrm{clk}_0 = \mathrm{clk}$		↑	↑	↑	↑	↑	↑	↑	↑
y_0	0	1	0	1	0	1	0	1	0
$\mathrm{clk}_1 = \overline{y_0}$	1	0	↑	0	↑	0	↑	0	↑
y_1	0	0	1	1	0	0	1	1	0
$\mathrm{clk}_2 = \overline{y_1}$	1	1	0	0	↑	1	0	0	↑
y_2	0	0	0	0	1	1	1	1	0
$y_2 y_1 y_0$	0	1	2	3	4	5	6	7	0

Analysis of the 3-bit ripple counter (Table 5.16); ↑: a rising edge

- y_0 flips at every clock edge, y_1 at every second one, y_2 at every fourth.
- Each stage clocks the next one: the clock edge *ripples* through the flip-flops. Hence *ripple counter*; and since they do not change at the same time, *asynchronous*.

The ripple



- Real flip-flops take a little time to change their output (drawn here much larger than it is). In a ripple counter these delays add up: y_1 changes one delay after y_0, y_2 one delay after y_1.
- With n flip-flops, the count is correct only n delays after the edge.

From 3 to 4, slowly



- 011 → 010: y_0 falls.
- 010 → 000: that clocks y_1, which falls.
- 000 → 100: that clocks y_2, which rises.
- For a moment the counter shows 2, then 0, before 4. A circuit that reads it too early gets a wrong number, like the glitches of `(Unit 9)`.

Synchronous or ripple?

	Synchronous	Ripple
Clock of flip-flop i	clk, for all	clk for the first, then $\overline{y_{i-1}}$
T inputs	$T_i = y_0 \cdots y_{i-1}$	all 1
Extra gates	a chain of AND gates	NOT gates only
Outputs change	together	one after another
Correct after an edge	one flip-flop delay	n delays
Wrong values on the way	no	yes

Remark

The ripple counter is the smaller circuit; the synchronous counter is the one the rest of a synchronous design can trust at every clock edge. The DX8's stepper uses the synchronous one.

Try it

1. A 3-bit up counter starts at 000. What does it show after 13 rising edges?
2. Which T inputs make a 4-bit synchronous up counter?
3. Make the synchronous counter count down.
4. Make the ripple counter count down.
5. In a 4-bit ripple counter, each flip-flop and its NOT gate take 10 ns. What happens on the step from 0111 to 1000?

Try it

1. A 3-bit up counter starts at 000. What does it show after 13 rising edges?
   101: 13 mod 8 = 5.
2. Which T inputs make a 4-bit synchronous up counter?
3. Make the synchronous counter count down.
4. Make the ripple counter count down.
5. In a 4-bit ripple counter, each flip-flop and its NOT gate take 10 ns. What happens on the step from 0111 to 1000?

Try it

1. A 3-bit up counter starts at 000. What does it show after 13 rising edges?
   101: 13 mod 8 = 5.
2. Which T inputs make a 4-bit synchronous up counter?
   $T_0 = 1$, $T_1 = y_0$, $T_2 = y_0 y_1$, $T_3 = y_0 y_1 y_2$.
3. Make the synchronous counter count down.
4. Make the ripple counter count down.
5. In a 4-bit ripple counter, each flip-flop and its NOT gate take 10 ns. What happens on the step from 0111 to 1000?

Try it

1. A 3-bit up counter starts at 000. What does it show after 13 rising edges?
   101: 13 mod 8 = 5.
2. Which T inputs make a 4-bit synchronous up counter?
   $T_0 = 1$, $T_1 = y_0$, $T_2 = y_0 y_1$, $T_3 = y_0 y_1 y_2$.
3. Make the synchronous counter count down.
   Bit i flips when all the bits below it are 0: $T_0 = 1$, $T_1 = \overline{y_0}$, $T_2 = \overline{y_0}\,\overline{y_1}$.
4. Make the ripple counter count down.

5. In a 4-bit ripple counter, each flip-flop and its NOT gate take 10 ns. What happens on the step from 0111 to 1000?

Try it

1. A 3-bit up counter starts at 000. What does it show after 13 rising edges?
   101: 13 mod 8 = 5.
2. Which T inputs make a 4-bit synchronous up counter?
   $T_0 = 1$, $T_1 = y_0$, $T_2 = y_0 y_1$, $T_3 = y_0 y_1 y_2$.
3. Make the synchronous counter count down.
   Bit i flips when all the bits below it are 0: $T_0 = 1$, $T_1 = \overline{y_0}$, $T_2 = \overline{y_0}\,\overline{y_1}$.
4. Make the ripple counter count down.
   Clock flip-flop i with y_{i-1} instead of $\overline{y_{i-1}}$: 000, 111, 110, …
5. In a 4-bit ripple counter, each flip-flop and its NOT gate take 10 ns. What happens on the step from 0111 to 1000?

Try it

1. A 3-bit up counter starts at 000. What does it show after 13 rising edges?
   101: 13 mod 8 = 5.
2. Which T inputs make a 4-bit synchronous up counter?
   $T_0 = 1$, $T_1 = y_0$, $T_2 = y_0 y_1$, $T_3 = y_0 y_1 y_2$.
3. Make the synchronous counter count down.
   Bit i flips when all the bits below it are 0: $T_0 = 1$, $T_1 = \overline{y_0}$, $T_2 = \overline{y_0}\,\overline{y_1}$.
4. Make the ripple counter count down.
   Clock flip-flop i with y_{i-1} instead of $\overline{y_{i-1}}$: 000, 111, 110, …
5. In a 4-bit ripple counter, each flip-flop and its NOT gate take 10 ns. What happens on the step from 0111 to 1000?
   0111, 0110, 0100, 0000, 1000: correct after 40 ns.

04

Summary

Summary

- An n-bit counter adds (or takes away) 1 at each clock edge and wraps around: it counts modulo 2^n.
- When counting up, bit i flips exactly when all the bits below it are 1.
- Synchronous counter: one clock for all, $T_i = y_0 \cdots y_{i-1}$; all bits change together.
- Ripple counter: every $T = 1$, each flip-flop clocked by $\overline{y_{i-1}}$; fewer gates, but the delays add up and wrong values appear on the way.
- Bit i of a counter divides the clock frequency by 2^{i+1}.

Before the next lecture

- Read Section 5.4 of the book.
- In Logisim-evolution, build both 3-bit counters from your T flip-flops of Lecture 12, with a reset, and check that they count from 0 to 7.
- Put a 3-to-8 decoder `(Unit 8)` after the synchronous counter: one output at a time is 1, in turn. That is the DX8's stepper `(Unit 21)`.

> **Next lecture** `(Unit 14)`
>
> Timing of sequential circuits: the setup and hold time constraints.